```
<Team name="Một anh cả ngã về M0">
    <Topic>XML & JSON</Topic>
    <Member>Lường Tiến Dũng</Member>
    <Member>Đồng Đức Dương</Member>
    <Member>Nguyễn Quốc Doanh</Member>
    <Member>Nguyễn Tùng Dương</Member>
</Team>
```

<data> { "format": "?" } </data>

XML & JSON

Hai định dạng lưu trữ và trao đổi dữ liệu — đặc điểm, công nghệ đi kèm và cách lựa chọn

PHẦN 1
Tổng quan XML
Đ.Đ.Dương · Slide 1–4

PHẦN 2
Các công nghệ XML
Đ.Đ.Dương · Slide 5–8

PHẦN 3
JSON
N.T.Dương · Slide 9–11

PHẦN 4
So sánh & Kết luận
N.T.Dương · Slide 12–14

XML — Extensible Markup Language

- **XML** = e**X**tensible **M**arkup **L**anguage
- Ngôn ngữ đánh dấu dùng để **lưu trữ, mô tả và trao đổi** dữ liệu
- Được chuẩn hóa bởi **W3C**
- Cho phép người dùng **tự định nghĩa tên thẻ**

> XML tập trung vào **cấu trúc và ý nghĩa** của dữ liệu, không trực tiếp quy định cách hiển thị.

Ví dụ:

```
<sinhVien>
    <hoTen>Luong Tien Dung</hoTen>
    <tuoi>41</tuoi>
</sinhVien>
```

Tên thẻ **sinhVien**, **hoTen**, **tuoi** do người viết tự đặt.

Những đặc điểm chính

01 Tự mô tả (Self-descriptive)

```
<hoTen>Nguyen Van A</hoTen>
```

→ Nhìn vào tên **hoTen** có thể hiểu dữ liệu đang biểu diễn gì.

02 Có cấu trúc phân cấp



→ Dữ liệu được tổ chức dưới dạng **cây.**

03 Có thể mở rộng

- Người dùng tự định nghĩa element
- Không bị giới hạn bởi bộ tag cố định

04 Độc lập nền tảng

- Có thể được xử lý bởi nhiều ngôn ngữ và hệ thống khác nhau.

Cấu trúc cơ bản

```xml
<?xml version="1.0" encoding="UTF-8"?>

<danhSachTruong>
    <truong>
        <maTruong>HUS</maTruong>
        <tenTruong>
            Đại học Khoa học Tự nhiên
        </tenTruong>
    </truong>
</danhSachTruong>
```

`<?xml …?>`	**XML Declaration**
`<danhSachTruong>`	**Root element**

`<truong>`
`<maTruong>`
`<tenTruong>`
	Các element con

`HUS, Đại học KHTN`	**Text / Content**

Element có thể chứa: Element khác · Text · Attribute

Các quy tắc cú pháp bắt buộc

01

Đóng thẻ đúng

```
<name>Dũng</name>
<!-- hoặc -->
<name/>
```

02

Lồng nhau đúng

✓ Đúng `<a><b>…</b></a>`

⚠ Sai `<a><b>…</a></b>`

03

Phân biệt chữ hoa / thường

`<Name> ≠ <name>`

04

Có đúng một root element

`<root> … </root>`

Toàn bộ tài liệu nằm trong root.

05

Attribute phải có giá trị

`<sinhVien id="001">`

Giá trị đặt trong dấu nháy.

⚠

Sai **một** quy tắc → tài liệu **không well-formed**, parser báo lỗi.

XML well-formed ≠ XML valid

WELL-FORMED

Đúng cú pháp XML

```
<student>
    <name>Dũng</name>
</student>
```

VALID

Đúng cú pháp + đúng bộ quy tắc

Tuân theo quy tắc về cấu trúc và dữ liệu, ví dụ:

```
student
  ├─── name  : string
  ├─── age   : integer
  └─── email : string
```



DTD / XML Schema (XSD)

DTD – Document Type Definition

- Element nào được phép xuất hiện
- Thứ tự các element
- Element xuất hiện bao nhiêu lần
- Attribute

XML Schema – XSD

Ngoài quy tắc cấu trúc, XSD còn hỗ trợ:

Kiểu dữ liệu: string, integer, date, boolean…

Min / Max

Độ dài

Pattern

Namespace

Ràng buộc phức tạp

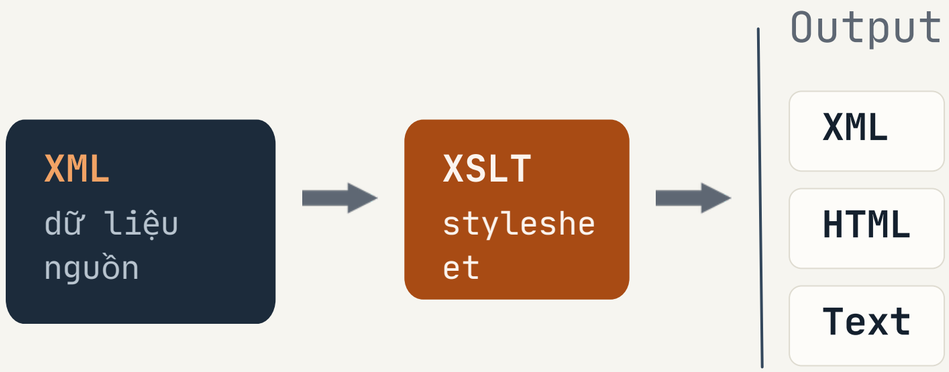
So sánh nhanh

Tiêu chí	DTD	XSD
Cấu trúc	Có	Có
Kiểu dữ liệu mạnh	Hạn chế	Có
Namespace	Hạn chế	Có
Ràng buộc phức tạp	Hạn chế	Có

Kết luận: XSD mạnh và linh hoạt hơn DTD.

XSLT — Chuyển đổi XML

XSLT = Extensible Stylesheet Language Transformations — biến đổi XML thành một dạng đầu ra khác.



Ví dụ ứng dụng

XML chứa dữ liệu sinh viên → **XSLT** → HTML hiển thị danh sách sinh viên

Điểm chính

- Tách dữ liệu khỏi cách trình bày
- Có thể lọc, sắp xếp và biến đổi dữ liệu
- Thường kết hợp với **XPath** để chọn node

Lưu ý: muốn xuất PDF thường cần thêm bước / công cụ định dạng hoặc chuyển đổi, không phải XSLT → PDF trực tiếp.

Truy vấn dữ liệu XML

XPATH Chọn node trong cây XML

```
<danhSachTruong>
    <truong>
        <maTruong>HUS</maTruong>
        <tenTruong>Đại học KHTN</tenTruong>
    </truong>
</danhSachTruong>
```

/danhSachTruong/truong/tenTruong

→ Lấy phần tử **tenTruong** (Đại học KHTN)

XQUERY Truy vấn mạnh hơn

Ngôn ngữ truy vấn XML, có thể:

- Tìm kiếm
- Lọc
- Sắp xếp
- Kết hợp dữ liệu XML
- Tạo kết quả mới

```
XPath  → chọn node
XQuery → truy vấn / xử lý phức tạp
```

JSON – JavaScript Object Notation

- Định dạng **trao đổi dữ liệu**
- Cú pháp **đơn giản, dễ đọc**
- Có nguồn gốc từ **JavaScript**
- Hiện nay **độc lập** với ngôn ngữ lập trình

Được sử dụng rộng rãi trong:

REST API | Web | Mobile | Hệ thống phân tán

Ví dụ

```
{
    "maTruong": "HUS",
    "tenTruong": "Đại học Khoa học Tự nhiên"
}
```

Cùng dữ liệu trường học như ví dụ XML, viết theo cặp **key : value**.

Các thành phần chính của JSON

Object – dùng { }

```
{
    "name": "Dung",
    "age": 20
}
```

→ Gồm các cặp

key : value

Array – dùng [ ]

```
{
    "students": [
        "An",
        "Binh",
        "Dung"
    ]
}
```

→ Biểu diễn **danh sách.**

Các kiểu dữ liệu

String	"HUS"
Number	20
Boolean	true / false
Null	null
Object	{ … }
Array	[…]

JSON ⇔ Object

JSON → Object

```
const obj = JSON.parse(json);
```

JSON string → JavaScript Object

Object → JSON

```
const json = JSON.stringify(obj);
```

JavaScript Object → JSON string

Trong Web → Đây là một lý do JSON rất phổ biến trong Web API.



So sánh XML và JSON

Tiêu chí	XML	JSON
Cú pháp	Nhiều tag	Gọn hơn
Kích thước	Thường lớn hơn	Thường nhỏ hơn
Cấu trúc	Element, Attribute, Namespace	Object, Array
Kiểu dữ liệu	Cơ bản, cần schema để ràng buộc	Có sẵn String, Number, Boolean, Null
Schema	DTD, XSD	JSON Schema
Truy vấn	XPath, XQuery	Thường xử lý qua API / ngôn ngữ
Transform	XSLT	Thường xử lý bằng code
Web API	Có sử dụng	Rất phổ biến
Cấu hình / tài liệu	Phù hợp trong nhiều hệ thống	Cũng có thể sử dụng

Lựa chọn định dạng

XML phù hợp khi

- Cần cấu trúc dữ liệu phức tạp
- Cần validation bằng XSD
- Cần namespace
- Cần công nghệ XML: XPath, XQuery, XSLT
- Làm việc với hệ thống / tiêu chuẩn đã dùng XML
- Hệ thống enterprise, tài liệu có cấu trúc, SOAP…

JSON phù hợp khi

- Xây dựng REST API
- Ứng dụng Web / Mobile
- Trao đổi dữ liệu giữa frontend và backend
- Muốn dữ liệu gọn và đơn giản
- Dữ liệu chủ yếu có dạng object / array

Không có định dạng nào luôn tốt hơn. Việc lựa chọn phụ thuộc vào yêu cầu của hệ thống.

Tổng kết

XML

Mạnh về **cấu trúc** và khả năng **mô tả dữ liệu**

- Element / Attribute
- Cấu trúc cây
- Well-formed
- XPath / XQuery
- XSLT
- DTD / XSD

JSON

Mạnh về **sự đơn giản** và **trao đổi dữ liệu**

- Object
- Array
- Kiểu dữ liệu cơ bản
- Phổ biến trong Web API
- Parse / stringify

XML và JSON phục vụ những nhu cầu khác nhau; **lựa chọn định dạng nên dựa trên yêu cầu của hệ thống.**

Q&A

Cảm ơn đã lắng nghe